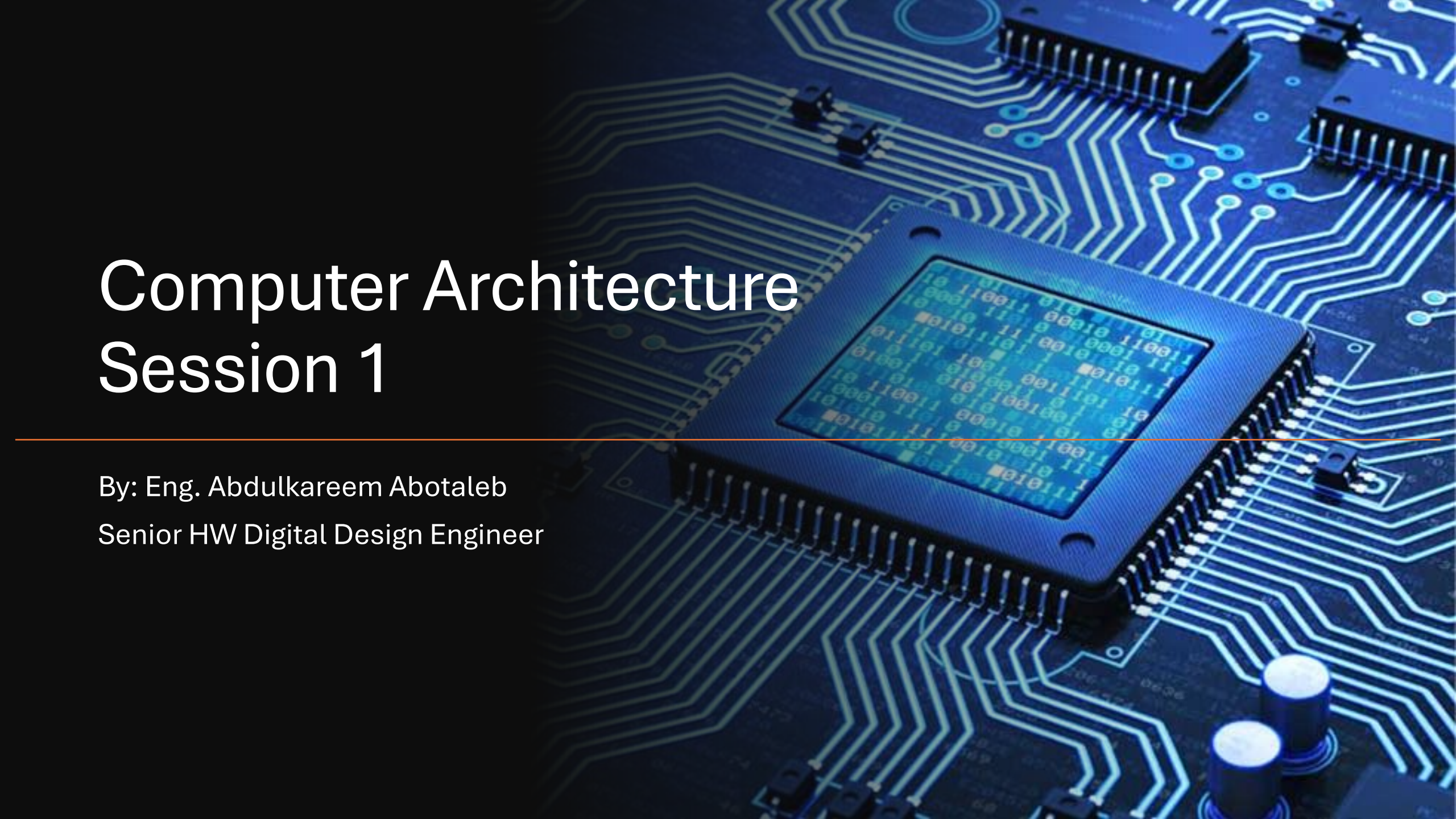
Computer Architecture Session 1

By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer



Roadmap [Phase #1: Single-Cycle MIPS]

#	Date	Tutorial	Practice Session	Week Work
1	19-Oct	**MIPS Architecture:** - Assembly Language. - Machine language. - Addressing Modes. **MIPS Microarchitecture:** - Singlecycle MIPS processor	Help in implementing the singlecycle datapath elements: Memory, register file, PC registers, MUXs. Help in implementing the control unit of the singlecycle MIPS processor.	Implement the basic (10 instructions) MIPS architecture verifying Test #1. Apply a solid testbench, which randomize the opcode and its operands, and finally monitor the output comparing with the expected changes (as a plus).
2	26-Oct	**MIPS Architecture:** - Programming (IF, For, While, Array, Functions). - Memory Map. - Starting a program.	Help in implementing some of the other instruction with some guidance.	Complete the remain instructions of MIPS architecture with a solid testbench in singlecycle processor. Apply Tests#2...6
3	2-Nov	**MIPS Architecture:** - Exceptions. - Advanced Instructions. **MIPS Microarchitecture:** - Multi-cycle MIPS processor.	Present the Multiplication and Division HW algorithms	Replace the normal multiplication and Division by the algorithms. Re-apply Test#6 and report the performance enhancement.

Roadmap [Phase #2 : Pipelined MIPS]

#	Date	Tutorial	Practice Session	Week Work
4	9-Nov	**MIPS Microarchitecture:** - Pipeline MIPS Processor. - Exceptions Implementation.	Present the basic implementation in pipeline microarchitecture solving their hazards.	Upgrade the single-cycle to pipelined processor solving all occurred hazards with a solid testbench. Apply Test#1…6
5	16-Nov	**MIPS Microarchitecture:** - Advanced Microarchitecture: * Deep Pipeline. * Branch Prediction. * Superscalar Processor. * Out-of-Order Processor. …	Present a proposed architecture for the Branch Prediction Unit.	Implement the exception unit [Co-processor 0]. Implement the Branch Prediction Unit. Apply Test#7-8
6	23-Nov	**Memory Systems:** - Cashes. - Virtual Memory.	Present a proposed architecture for the set-associative cache.	Implement the et-associative cache and integrate it to the MIPS implementation [as a plus].

Roadmap [Phase #3: SoC MIPS-based]

#	Date	Tutorial	Practice Session	Week Work
7	30-Nov	**SoC Architecture:** - System Architecture. - AMBA protocol (AHB). - AHB address decoder. - GPIO.	Highlight how to add AHB access to MIPS and memories, and how to integrate the AHB decoder and GPIO.	Merge AHB master to MIPS, Merge AHB slave to memory. Integrate the MIPS, AHB decoder, memories and GPIO together with AHB protocol.
8	7-Dec	**SoC Architecture:** - AMBA protocol (APB). - AHB2APB Bridge. - Serial Communication: * UART, SPI, I2C	Highlight how to integrate the AHB2APB bridge and the APB peripherals.	Integrate the AHB2APB bridge, TIMER and UART together.
9	14-Dec	**FPGA:** - Area, Timing, Power	Some tips for applying the pipelined SoC on the FPGA.	Apply the pipelined SoC on the FPGA.
10	21-Dec	_No_Tutorial_	Buffer for any shifted topic.	Buffer for any lated delivery.

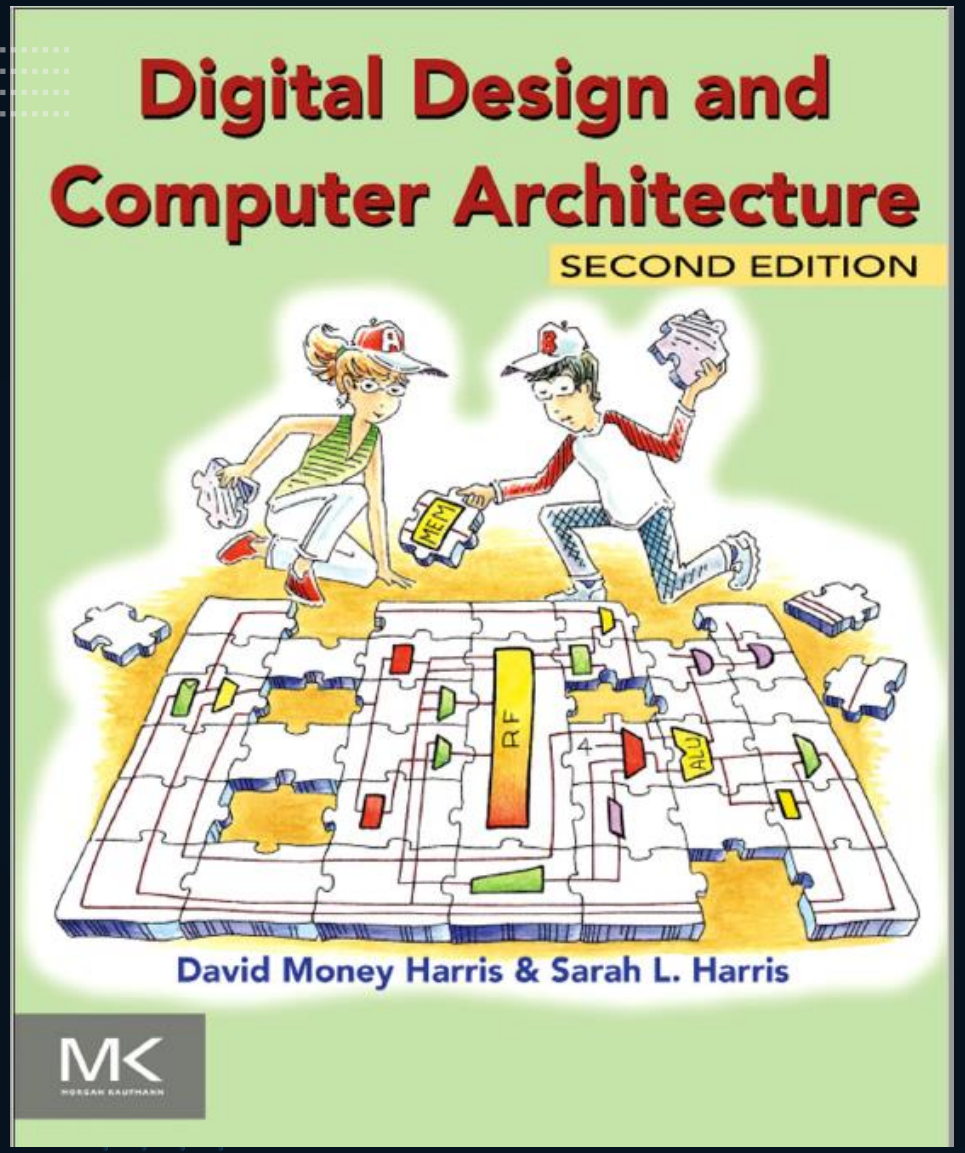
Policy to obtain the Certificate:

Your Grade:

- 70% of the grade: Achieve all three phases of our project.
  - Phase #1: pass Tests#1…7 and complete the performance sheet [20%].
  - Phase #2: pass Tests#1…8 and complete the performance sheet [30%].
  - Phase #3: pass Tests#9-10 and complete the performance sheet [20%].
- 15% of the grade: Commit to the deadline of the weekly task.
- 15% of the grade: Attend 70% of our sessions at least [Active Attendance].

Ethics Agreement:

- Don’t pass the material or the session records out our group.
- Don’t leave the online session without permission [It’s better to request a break].



This presentation is prepared from:

Digital Design and Computer Architecture, 2nd Edition.
David Harris and Sarah L. Harris.

Agenda

Chapter 6: Architecture

- # 1. Introduction
- # 2. Assembly Language
- # 3. Machine Language
- # 5. Addressing Modes

Chapter 7: Microarchitecture

- # 1. Introduction
- # 2. Performance Analysis
- # 3. Single-Cycle Processor

1. Introduction

Introduction

- The architecture is defined by the **instruction set** (language) and **operand locations** (registers and memory).
- Many different architectures exist such as x86, MIPS, SPARC, and PowerPC. → IOS
- The first step is to learn the computer language: its words "**instructions**", and its dictionary "**instruction set**".
- All programs, even the complex applications, use the same instruction set as a series of simple instructions such as add, subtract, and jump.
- The instructions indicate both the operation to perform and the operands to use.
- The operands may come from memory, from registers, or from the instruction itself.

Introduction

- Computer hardware understands only 1's and 0's, so instructions are encoded as binary numbers in a **machine language** format.
- We represent the instructions in a symbolic **assembly language** format to be more readable.
- Almost all architectures define **basic instructions**, such as add, subtract, and jump, that operate on memory or registers.
  Once you have learned one instruction set, understanding others is straightforward.

Introduction

- A **computer architecture** does not define the hardware implementation.
- For example, Intel and AMD sell different microprocessors with the same x86 architecture. They all can run the same programs with different hardware, so they have trade-offs in performance, price, and power.
- The **microarchitecture** defines the specific arrangement of registers, memories, ALUs, and other building blocks to form the microprocessor.
- Our training introduces **the MIPS architecture** that was first developed by John Hennessy at Stanford in the 1980s. MIPS processors are used by Silicon Graphics, Nintendo, and Cisco.

2. Assembly Language

Agenda

- # 2. Assembly Language
  - # Instructions
  - # Operands: Registers, Memory, and Constants

ASSEMBLY LANGUAGE

- Assembly language is the human-readable representation of the computer language.
- Each assembly language instruction specifies both the operation to perform and the operands which to operate on.

- We introduce simple arithmetic instructions and show how these operations are written in assembly language.
- We will define the MIPS instruction operands: registers, memory, and constants.

Instructions

Design Principle 1: **Simplicity favors regularity.**

- Instructions with *a consistent number of operands* are easier to encode and handle in hardware.
- Example shows a code for adding and subtracting two variables b and c and writing the result to a.
- The first part of the assembly instruction, **add** or **sub** indicates which operation to perform.

	High-Level Code	MIPS Assembly Code
Addition	a = b + c;	add a, b, c
Subtraction	a = b − c;	sub a, b, c

Instructions

Design Principle 2: **Make the common case fast.**

- The MIPS instruction set makes the common case fast by applying the reduced instruction set computer (RISC) architecture.

	instructions	hardware complexity	Req SW app	Example
RISC	Simpler	Simpler	Many instructions	MIPS
CISC	More Complex	More Complex	Less instructions	Intel x86

- For example, x86 defines a “string move” instruction that copies a string (a series of characters) requiring many simple instructions in a RISC machine. However, the cost of implementing complex instructions is added that slows down the simple instructions.

Instructions

- Example
- More complex high-level code translates into multiple MIPS instructions.
- The assembly language program in this example requires **a temporary variable** to store the intermediate result. We have to use multiple assembly language instructions to perform more complex operations.

High-Level Code	MIPS Assembly Code
a = b + c – d;	sub t, c, d # t = c – d add a, b, t # a = b + t

Operands: Registers, Memory, and Constants

- **An instruction operates on operands**. In Example the variables a, b, and c are all operands. But computers need **a physical location** from which to store the binary data.
- Operands can be stored in **registers** or **memory**, or they may be **constants** stored in the **instruction** itself. Computers use various locations to hold operands to optimize the speed and data size.
- Operands stored as constants or in registers are **accessed quickly**, but they hold only a **small** amount of data.
- Additional data must be accessed from memory, which is **large** but **slow**.
- MIPS is called a 32-bit architecture because it operates on 32-bit data.

Operands: Registers, Memory, and Constants

Registers

Design Principle 3: **Smaller is faster**.

- Most architectures specify a small number of registers that hold commonly used operands so that they can run fast.
- The MIPS architecture uses 32 registers, called the register set or register file.
- The fewer the registers, the faster they can be accessed.
- A small register file is typically built from a small SRAM array which uses a small decoder and bit lines connected to relatively few memory cells, so it has a shorter critical path than a large memory does.

Operands: Registers, Memory, and Constants

- Example: the add instruction is shown with register operands. MIPS register names are replaced by $s0, $s1, and $s2.

High-Level Code	MIPS Assembly Code
a = b + c;	# $s0 = a, $s1 = b, $s2 = c add $s0, $s1, $s2 # a = b + c

- The instruction adds the 32-bit values contained in $s1 (b) and $s2 (c) and writes the 32-bit result to $s0 (a).
- MIPS generally stores variables in 18 of the 32 registers: $s0–$s7, and $t0–$t9.
- Register names beginning with $s are called saved registers which store variables such as a, b, and c.

Operands: Registers, Memory, and Constants

- Register names beginning with $t are called temporary registers. They are used to store temporary variables.
- Example: Using a temporary register, $t0, to store the intermediate calculation of c – d.

High-Level Code	MIPS Assembly Code
a = b + c – d;	# $s0 = a, $s1 = b, $s2 = c, $s3 = d sub $t0, $s2, $s3 # t = c – d add $s0, $s1, $t0 # a = b + t

Operands: Registers, Memory, and Constants

Example 1 Translating High-level Code to Assembly Language

- Translate the following high-level code into assembly language.
- Assume variables a–c are held in registers $s0–$s2 and f–j are in $s3–$s7.

```
a = b – c;
f = (g + h) – (i + j);
```

Solution:

```
# MIPS assembly code
# $s0 = a,      $s1 = b,      $s2 = c,
# $s3 = f,      $s4 = g,      $s5 = h
# $s6 = i,      $s7 = j

sub $s0, $s1, $s2        # a = b – c
add $t0, $s4, $s5        # $t0 = g + h
add $t1, $s6, $s7        # $t1 = i + j
sub $s3, $t0, $t1        # f = (g + h) – (i + j)
```

Operands: Registers, Memory, and Constants

The Register Set

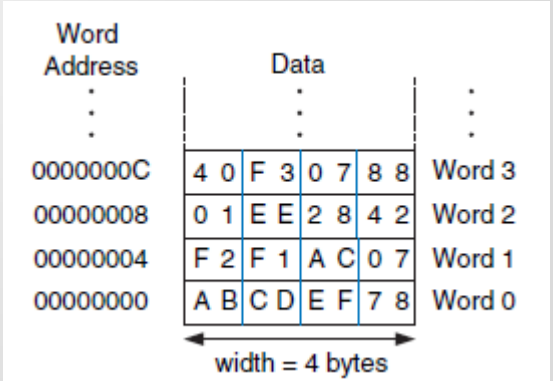
- The MIPS architecture defines 32 registers.
- Each register has a name and a number ranging from 0 to 31.
- $0 always contains the value 0 because this constant is so frequently used in computer programs.
- The remaining registers will be described throughout this chapter.

Name	Number	Use
$0	0	the constant value 0
$at	1	assembler temporary
$v0–$v1	2–3	function return value
$a0–$a3	4–7	function arguments
$t0–$t7	8–15	temporary variables
$s0–$s7	16–23	saved variables
$t8–$t9	24–25	temporary variables
$k0–$k1	26–27	operating system (OS) temporaries
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	function return address

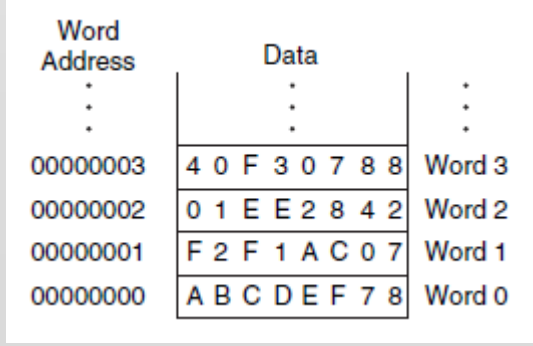
Operands: Registers, Memory, and Constants

Memory

- We should use memory in order to store the data, because of the less space in registers.
- The memory is large and slow; however, the register file is small and fast. So, commonly used variables are kept in registers. By using a combination of memory and registers, a program can access a large amount of data quickly.
- The MIPS architecture uses **32-bit memory addresses** and **32-bit data words**.



- **MIPS** uses **a byte-addressable memory**:
  Each byte in the memory has a unique address.



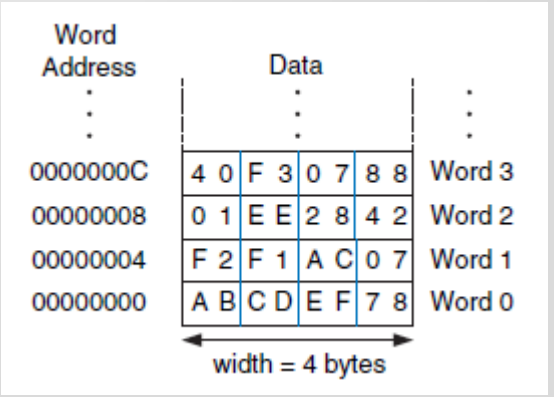
- Second figure shows **a word-addressable memory**: Each 32-bit data word has a unique 32-bit address.

Operands: Registers, Memory, and Constants

Memory

- MIPS uses the load word instruction, **lw**, to read a data word from memory into a register, and the store word instruction, **sw**, to write a data word from a register into memory.
- The two instructions specify the effective address in memory as the sum of **a base address** (inside the parentheses) and **an offset** (before the parentheses).
- Example: The MIPS assembly code reads words 0, 2, and 3 and writes words 1, 8, and 100. The offset can be written in decimal or hexadecimal.

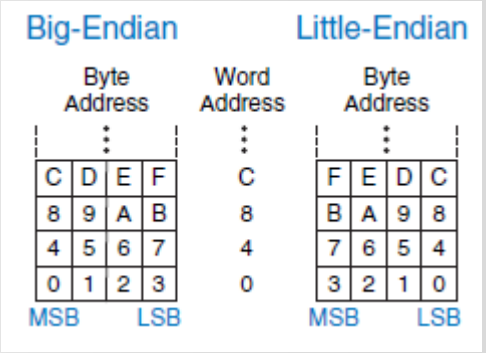
```
lw $s0, 0($0)        # read data word 0 (0xABCDEF78) into $s0
lw $s1, 8($0)        # read data word 2 (0x01EE2842) into $s1
lw $s2, OxC($0)      # read data word 3 (0x40F30788) into $s2
sw $s3, 4($0)        # write $s3 to data word 1
sw $s4, 0x20($0)     # write $s4 to data word 8
sw $s5, 400($0)      # write $s5 to data word 100
```



Operands: Registers, Memory, and Constants

Memory

- MIPS also provides the **lb** and **sb** instructions that load and store single bytes in memory.
- Byte-addressable memories classified to:
  Big-endian format: bytes are numbered starting with 0 at the MSB.
  Little-endian format: bytes are numbered starting with 0 at LSB.
  Word addresses are the same in both formats.



- IBM's PowerPC (found in Macintosh computers) uses big endian addressing.
  Intel's x86 architecture (found in PCs) uses little-endian addressing.
  Some MIPS processors are little-endian, and some are big-endian.
- In our training, we will use little-endian format whenever byte ordering matters.
- In the MIPS architecture, word addresses for lw and sw must be word aligned, so the address must be divisible by 4. Thus, the instruction lw $s0, 7($0) is an **illegal** instruction.

Operands: Registers, Memory, and Constants

Constants/Immediates

- Constants are immediately available from the instruction and do not require a register or memory access.
- For example, Add immediate, **addi**, is another common MIPS instruction that adds the immediate specified in the instruction to a value in a register.

High-Level Code	MIPS Assembly Code
a = a + 4; b = a − 12;	# $s0 = a, $s1 = b addi $s0, $s0, 4 # a = a + 4 addi $s1, $s0, −12 # b = a − 12

- The immediate specified in an instruction is a 16-bit two's complement number in the range [–32,768, 32,767].
- There is no **subi** instruction in the MIPS architecture because of the simplicity as the subtraction is equivalent to adding a negative number.
- The lw and sw instructions use two register operands and a constant which violates design principle 1, simplicity favors regularity, introducing the last design principle.

Operands: Registers, Memory, and Constants

Constants/Immediates

Design Principle 4: Good design demands good compromises.

- A single instruction format would be simple but not flexible.

- The MIPS instruction set makes the compromise of supporting three instruction formats:

  1- R-type instructions operate on three registers : such as add and sub.
  2- I-type instructions operate on two registers and a 16-bit immediate: such as lw and addi.
  3- J-type instructions operate on one 26-bit immediate: such as jump.

3. Machine Language

Agenda

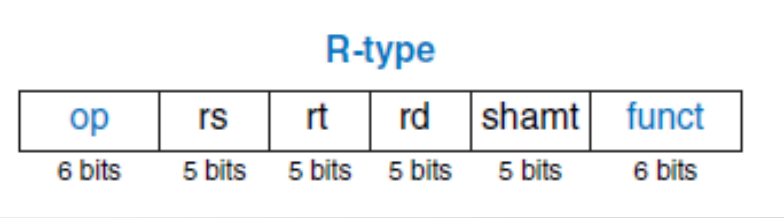
- # 3. Machine Language
  - # R-Type Instructions
  - # I-Type Instructions
  - # J-Type Instructions
  - # Interpreting Machine Language Code
  - # The Power of the Stored Program

3. Machine Language

- Digital circuits understand only 1's and 0's. So, a program written in assembly language is translated to a representation using only 1's and 0's (**machine language**).
- MIPS uses 32-bit instructions. All instructions are stored as words in the memory as **simplicity favors regularity** even the instructions which doesn't require all 32 bits.
- Simplicity would also encourage a single instruction format, which is too restrictive. MIPS makes the compromise of defining three instruction formats: R-type, I-type, and J-type.
- This small number of formats allows for **some regularity** among all the types, and thus simpler hardware, with serving different instruction needs, such as the need to encode large constants in the instruction.
  R-type instructions operate on three registers.
  I-type instructions operate on two registers and a 16-bit immediate.
  J-type (jump) instructions operate on one 26-bit immediate.

3.1 R-Type Instructions

- R-type (register-type) instructions use three registers as operands: two as sources, and one as a destination.



- The 32-bit instruction has six fields: op, rs, rt, rd, shamt, and funct.
- All R-type instructions have op (opcode or operation code) of 0.
- The specific R-type operation is determined by the funct (the function) field.
- For example, the opcode and funct fields for the add instruction are 0 (000000_2) and 32 (100000_2), respectively. Similarly, the sub instruction has an opcode and funct field of 0 and 34.
- The operands are encoded in the three fields: **rs**, **rt**, and **rd**.
  The first two registers, **rs** and **rt**, are the source registers; **rd** is the destination register.

3.1 R-Type Instructions

- The fifth field, shamt, is used only in shift operations as the shift amount value is stored in the 5-bit shamt field.
- For all other R-type instructions, shamt is 0.
- For example, the assembly instruction add $s0, $s1, $s2 has rs = $s1 (17), rt = $s2 (18), and rd = $s0 (16).

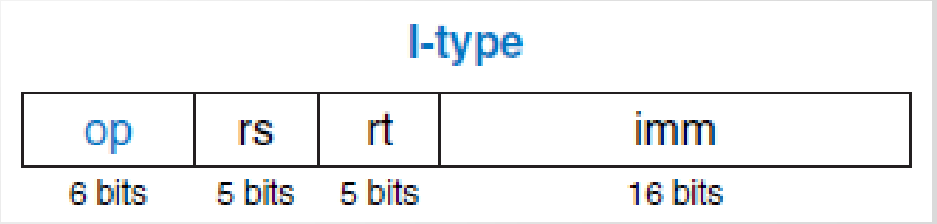
Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
add $s0, $s1, $s2	0	17	18	16	0	32	000000	10001	10010	10000	00000	100000	(0x02328020)
sub $t0, $t3, $t5	0	11	13	8	0	34	000000	01011	01101	01000	00000	100010	(0x016D4022)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

- The **destination** is the first register in an assembly language instruction, but it is the third register field (rd) in the machine language instruction.

Name	Number	Use
$0	0	the constant value 0
$at	1	assembler temporary
$v0–$v1	2–3	function return value
$a0–$a3	4–7	function arguments
$t0–$t7	8–15	temporary variables
$s0–$s7	16–23	saved variables
$t8–$t9	24–25	temporary variables
$k0–$k1	26–27	OS temporaries
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	function return address

3.2 I-Type Instructions

- I-type (immediate-type) instructions use two register operands and one immediate operand.



- The 32-bit instruction has four fields: op, rs, rt, and imm.
  The first three fields: op, rs, and rt, are like those of R-type instructions.
  The imm field holds the 16-bit immediate.
- The operation is determined by the opcode.
  rs and imm are always used as source operands.
  rt is used as a destination for some instructions (as addi and lw) but as another source for others (as sw).

3.2 I-Type Instructions

- Example of encoding I-type instructions.

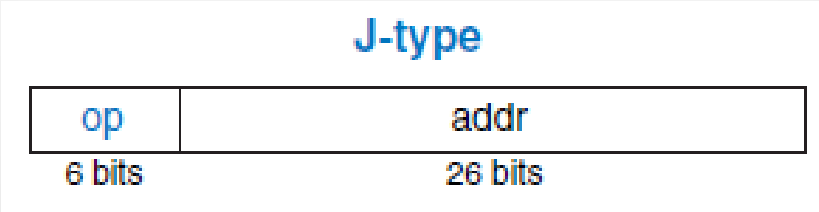
Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
addi $s0, $s1, 5	8	17	16	5	001000	10001	10000	0000 0000 0000 0101	(0x22300005)
addi $t0, $s3, –12	8	19	8	–12	001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
lw $t2, 32($0)	35	0	10	32	100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
sw $s1, 4($t1)	43	9	17	4	101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

- In I-type instructions, a 16-bit immediate is used in 32-bit operations. Most MIPS instructions (addi, lw, and sw) apply sign extension by placing 0's in the upper half for the positive immediates, while placing 1's for the negative immediates.
- The logical operations (andi, ori, xori) apply zero extension by placing 0's in the upper half.

Name	Number	Use
$0	0	the constant value 0
$at	1	assembler temporary
$v0–$v1	2–3	function return value
$a0–$a3	4–7	function arguments
$t0–$t7	8–15	temporary variables
$s0–$s7	16–23	saved variables
$t8–$t9	24–25	temporary variables
$k0–$k1	26–27	OS temporaries
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	function return address

3.3 J-Type Instructions

- J-type (jump-type) format is used only with jump instructions.

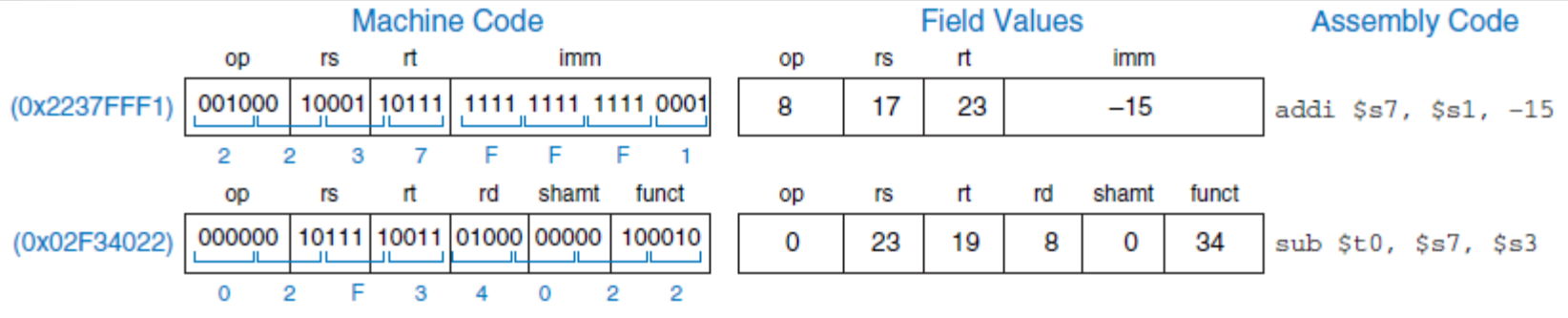


- This instruction format uses a single 26-bit address operand, **addr**.
- Like other formats, J-type instructions begin with a 6-bit opcode.
- The remaining bits are used to specify an address, addr.

3.4 Translating Machine Language Code

- To interpret machine language, one must decode the fields of each 32-bit instruction word.
- Different instructions use different formats, but all formats start with a 6-bit opcode field.
- If the opcode is 0, the instruction is R-type; otherwise, it's I-type or J-type.
- Example:

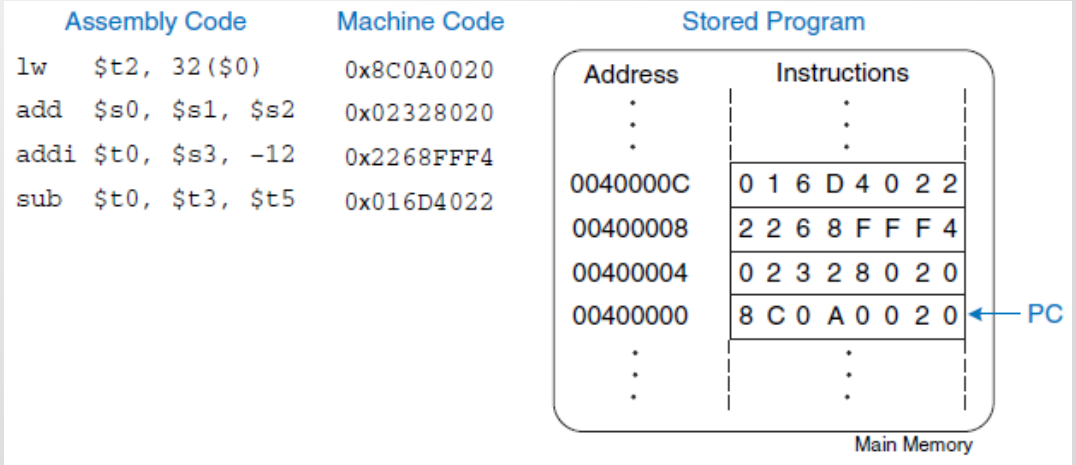
  Translate the following machine language code into assembly language.
  0x2237FFF1
  0x02F34022



3.5 The Power of the Stored Program

- **Stored program concept** means a series of instructions in the memory.
- This is why computers are so **powerful**, as running different programs don't require to reconfigure the hardware; it only requires writing the new program to the memory.

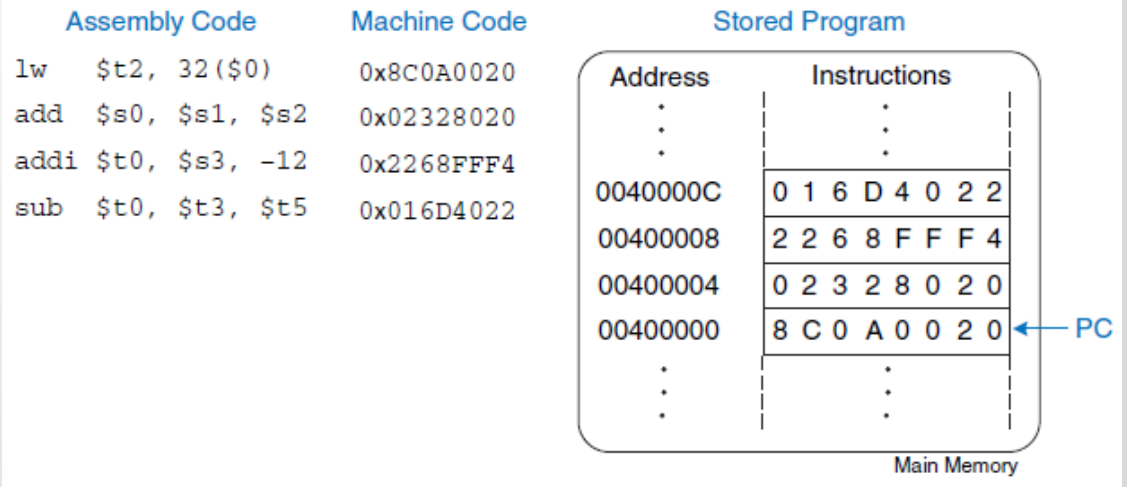
- In MIPS programs, the instructions are normally stored starting at address 0x00400000.
- To run or execute the stored program, the processor fetches the instructions from memory sequentially.
- The fetched instructions are then decoded and executed by the digital hardware.



- The address of the current instruction is kept in a 32-bit register called the program counter (PC).

3.5 The Power of the Stored Program

- To execute the code, the operating system sets the PC to address 0x00400000.
- The processor reads the instruction at that memory address and executes the instruction, 0x8C0A0020.
- The processor then increments the PC by 4 to 0x00400004, fetches and executes that instruction, and repeats.



- The **architectural state** of a microprocessor holds the state of a program.
- For MIPS, the architectural state consists of **the register file** and **PC**.
- The OS saves the architectural state at some point in the program, and interrupt the program to do something else, then restore the state such that the program continues properly.

5. Addressing Modes

5. Addressing Modes

- MIPS has 5 modes: register-only, immediate, base, PC-relative, and pseudo-direct.
- The first three modes define modes of reading and writing *operands*.
- The last two (PC-relative and pseudo-direct) define modes of writing the *PC*.

Register-Only Addressing

- It uses registers for all source and destination operands, like all R-type instructions

Immediate Addressing

- It uses the 16-bit immediate along with registers as operands, like addi and lo lui.

Base Addressing

- Memory access instructions, such as lw and sw, use base addressing.
- The effective address of the memory operand is found by adding the base address in register rs to the sign-extended 16-bit offset found in the immediate field.

5. Addressing Modes

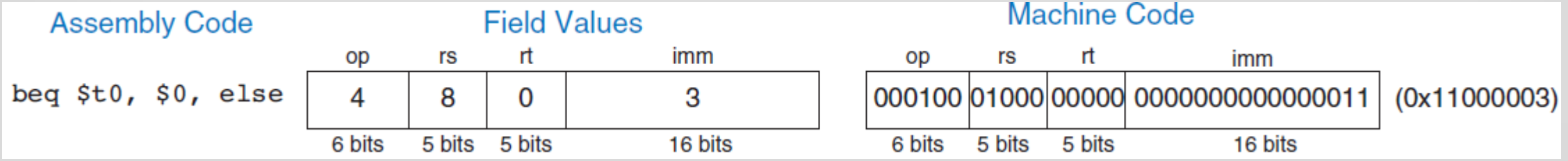
PC-Relative Addressing

- It's used in *conditional branch instructions* to specify the new PC if the branch is taken.
- The signed offset in the immediate field is added to the PC to obtain the new PC; hence, the branch destination address is said to be relative to the current PC.
- The branch target address (BTA) is the address of the next instruction to execute if the branch is taken.

MIPS Assembly Code	
0xA4	**beq** $t0, $0, else
0xA8	addi $v0, $0, 1
0xAC	addi $sp, $sp, 8
0xBO	jr $ra
0xB4	else: addi $a0, $a0, −1
0xB8	jal factorial

Example: The machine code for the beq instruction.

- The beq instruction has a BTA of 0xB4, and PC+4 (0xA8).
- So, the value in the immediate field of beq is 3.

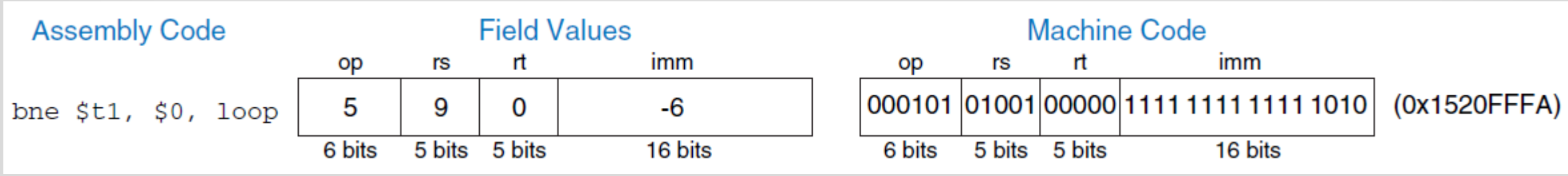


5. Addressing Modes

PC-Relative Addressing

Example: Calculate the immediate field and show the machine code for the bne instruction.

MIPS Assembly Code	
0x40	loop: add $t1, $a0, $s0
0x44	lb $t1, 0($t1)
0x48	add $t2, $a1, $s0
0x4C	sb $t1, 0($t2)
0x50	addi $s0, $s0, 1
0x54	**bne** $t1, $0, loop
0x58	lw $s0, 0($sp)



5. Addressing Modes

Pseudo-Direct Addressing

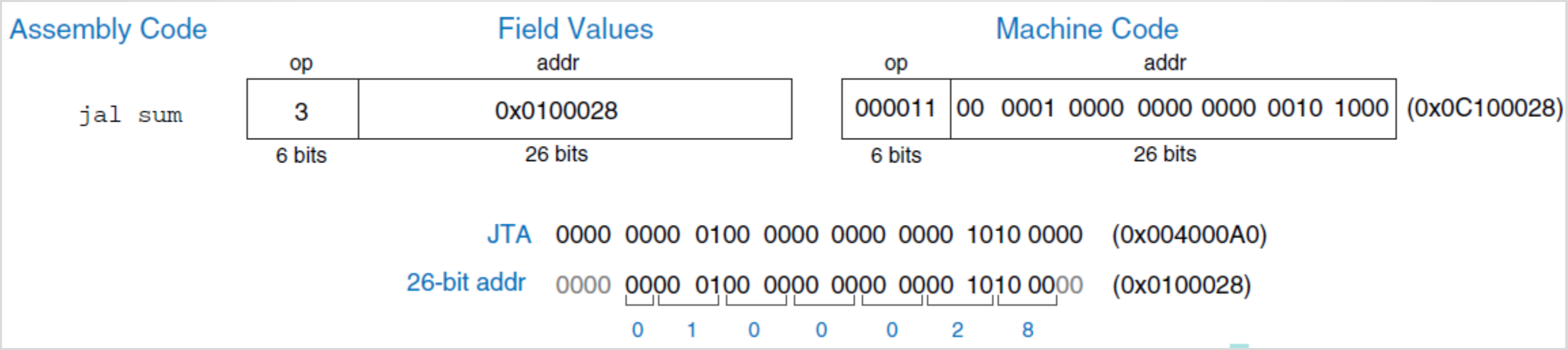
- It specifies the address of the jump instructions, **j** and **jal**.
- The 26-bits address in the instruction specifies the jump target address JTA (27:2).
- The four most significant bits, JTA(31:28), are obtained from the four most significant bits of PC + 4.
- The two least significant bits, JTA(1:0), should always be 0, as instructions are word aligned.
- Because the 4 most significant bits of JTA are taken from PC+4, the jump range is limited.
- Note that the jump register instruction, jr, is not a J-type instruction.
  It is an R-type instruction that jumps to the 32-bit value held in register rs.

5. Addressing Modes

Pseudo-Direct Addressing

- Example: a jal instruction using pseudo-direct addressing.

MIPS Assembly Code	
0x0040005C	jal sum
…	
0x004000A0	sum: add $v0, $a0, $a1



Agenda

Chapter 7: Microarchitecture

1. Introduction
2. Performance Analysis
3. Single-Cycle Processor
4. Multi-Cycle Processor
5. Pipelined Processor
6. Exceptions

1. Introduction

1. Introduction

- **Microarchitecture** is the connection between logic and architecture.
- Microarchitecture is the specific arrangement of registers, ALUs, finite state machines (FSMs), memories, and other logic building blocks.
- A particular architecture, such as MIPS, may have many different microarchitectures, each with different trade-offs of performance, cost, and complexity.
- They all run the same programs, but their internal designs vary widely.
- We will introduce the design of 3 different microarchitectures to illustrate the trade-offs.

1. Introduction

1.1 Architectural State and Instruction Set

1.2 Design Process

1.3 MIPS Microarchitectures

1. Introduction

1.1 Architectural State and Instruction Set

- The computer architecture is defined by its instruction set and architectural state.
- The architectural state for the MIPS processor consists of the program counter and the 32 registers.
- Based on the current architectural state, the processor executes a particular instruction with a particular set of data to produce a new architectural state.
- Some microarchitectures contain additional non-architectural state to either simplify the logic or improve performance.

1. Introduction

1.1 Architectural State and Instruction Set

- we will start implementing the following basic instructions to keep the microarchitectures easy to understand:
    - R-type arithmetic/logic instructions: add, sub, and, or, slt
    - Memory instructions: lw, sw
    - Branches: beq
- After building the microarchitectures with these instructions, we extend them to handle addi and j.
- Once you understand how to implement these instructions, you can expand the hardware to handle others.

1. Introduction

1.1 Architectural State and Instruction Set

1.2 Design Process

1.3 MIPS Microarchitectures

1. Introduction

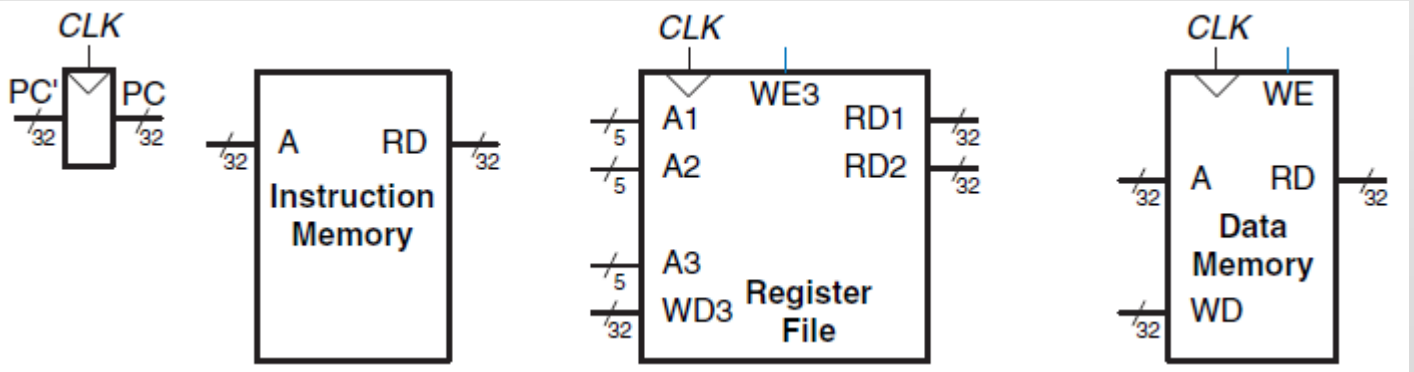
1.2 Design Process

- We will divide our microarchitectures into two parts: the datapath and the control.
- The datapath operates on words of data. It contains structures such as memories, registers, ALUs, and multiplexers. MIPS is a 32-bit architecture (32-bit datapath).
- The control unit receives the current instruction from the datapath and produces **multiplexer select**, **register enable**, and **memory write** signals to control the operation of the datapath.
- Start with hardware containing the memories and the architectural state (the program counter and registers). Then, add blocks of combinational logic between the state elements to compute the new state based on the current state.

1. Introduction

1.2 Design Process

- It is often convenient to partition the overall memory into two smaller memories, one containing instructions and the other containing data.
- The block diagram has four state elements: the program counter, register file, and instruction and data memories.
- State elements usually have a reset input to put them into a known state at start-up.



1. Introduction

- The **program counter** is an ordinary 32-bit register. Its output (PC) points to the current instruction address. Its input (PC') indicates the next instruction address.
- The **instruction memory** has a single read port. It takes a 32-bit instruction address input, A, and reads the 32-bit data (instruction) onto the read data output, RD.
- The 32-element **register file** (32-bit) has two read ports and one write port.
  The read ports take 5-bit address inputs (A1 and A2) and read the 32-bit register values onto read data outputs RD1 and RD2, respectively.
  The write port takes a 5-bit address input (A3), a 32-bit write data input (WD), a write enable input (WE3), and a clock.
- The **data memory** has a single read/write port.
  If the write enable (WE) is 1, it writes data (WD) into address (A) on the rising edge of the clock. Otherwise, it reads address A onto RD.

1. Introduction

- The state elements (instruction memory, register file, and data memory) are **all read combinationally**.
  If the address changes, the new data appears at RD after some propagation delay; no clock is involved.
- However, writing the state elements is only on the rising edge of the clock, so they are **synchronous sequential circuits**.
- The address, data, and write enable must **setup** sometime before the clock edge and must remain stable until a **hold** time after the clock edge.

1. Introduction

1.1 Architectural State and Instruction Set

1.2 Design Process

1.3 MIPS Microarchitectures

1. Introduction

1.3 MIPS Microarchitectures

- We develop three microarchitectures for the MIPS processor architecture: single-cycle, multicycle, and pipelined. They differ in the connection between the state elements and the amount of nonarchitectural state.

1) The single-cycle microarchitecture

- It executes an entire instruction in one cycle.
- It is easy to explain and has a simple control unit.
- It doesn’t require any nonarchitectural state, as it’s done in a cycle.
- The cycle time is limited by the slowest instruction.

1. Introduction

1.3 MIPS Microarchitectures

2) The multicycle microarchitecture

- It executes instructions in a series of shorter cycles.
- Simpler instructions execute in fewer cycles than complicated ones.
- It reduces the HW cost by reusing expensive hardware blocks such as adders and memories.
- It needs to add several nonarchitectural registers to hold intermediate results.
- It executes only one instruction at a time, but each instruction takes multiple clock cycles.

1. Introduction

3) The pipelined microarchitecture

- It applies pipelining to the single-cycle microarchitecture.
- It can execute several instructions simultaneously, improving the throughput significantly.
- Pipelining must add logic to handle dependencies between simultaneously executing instructions.
- It also requires nonarchitectural pipeline registers.

2. Performance Analysis

2. Performance Analysis

- Performance can be measured by the total execution time of a collection of programs **(called benchmarks)** that will often be run.
- The execution times of **Benchmark** programs are commonly published to give some indication of how a processor performs.
- The execution time of a program, measured in seconds, is given by:

$$\text{Execution Time} = \#\text{instructions} * \frac{\text{cycles}}{\text{instruction}} * \frac{\text{seconds}}{\text{cycle}}$$

2. Performance Analysis

- The number of instructions in a program depends on the processor architecture. Some architectures have complicated instructions that do more work per instruction, thus reducing the number of instructions in a program. However, these complicated instructions are often slower to execute in hardware.
- Assume that we are executing known programs on a MIPS processor, so the number of instructions for each program is constant, independent of the microarchitecture.

- The number of cycles per instruction, often called CPI, is the number of clock cycles required to execute an average instruction.
- It is the reciprocal of the throughput (instructions per cycle, or IPC).
- Different microarchitectures have different CPIs.

2. Performance Analysis

- The number of seconds per cycle is the clock period, T_c.
- The clock period is determined by the critical path through the logic on the processor.
- Logic and circuit designs also significantly affect the clock period.
  For example, a carry-lookahead adder is faster than a ripple-carry adder.
- Manufacturing advances have historically doubled transistor speeds every 4–6 years, so a microprocessor built today will be much faster than one from last decade, even if the microarchitecture and logic are unchanged.
- The challenge of the microarchitect is to choose the design that minimizes the execution time while satisfying constraints on cost and/or power consumption.

3. Single-Cycle Processor

3. Single-Cycle Processor

- Single-Cycle MIPS microarchitecture executes any instruction in a single cycle.
- We begin constructing the datapath by connecting the state elements with combinational logic that can execute the various instructions.
- Control signals determine which specific instruction is carried out by the datapath at any given time.
- The controller contains combinational logic that generates the appropriate control signals based on the current instruction.
- We conclude by analyzing the performance of the single-cycle processor.

3. Single-Cycle Processor

3.1 Single-Cycle Datapath

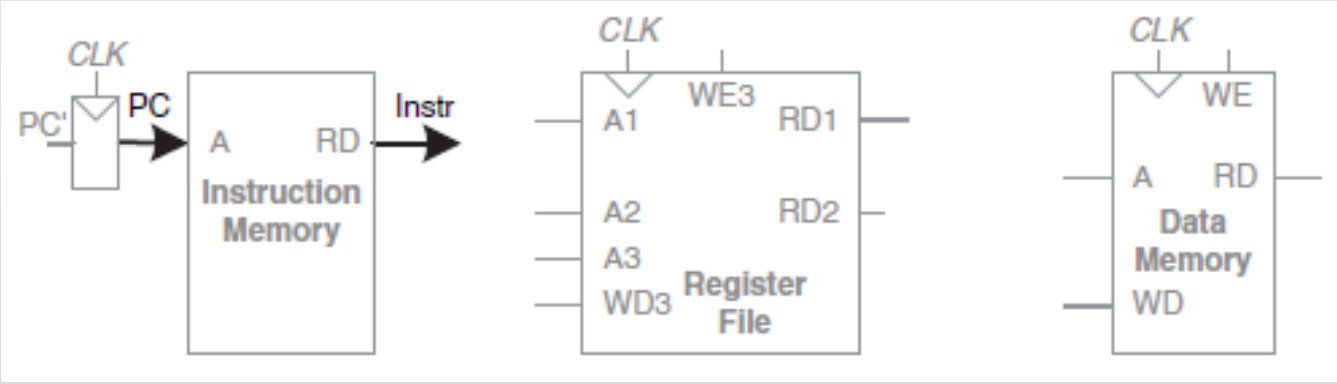
3.2 Single-Cycle Control

3.3 More Instructions

3.4 Performance Analysis

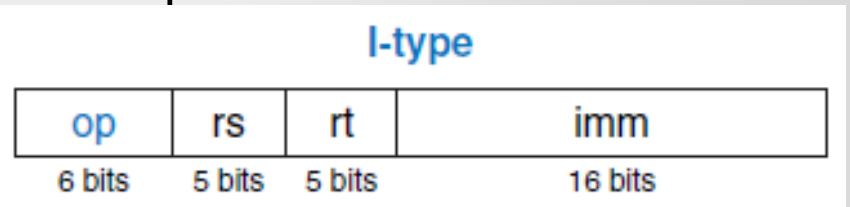
3. Single-Cycle Processor

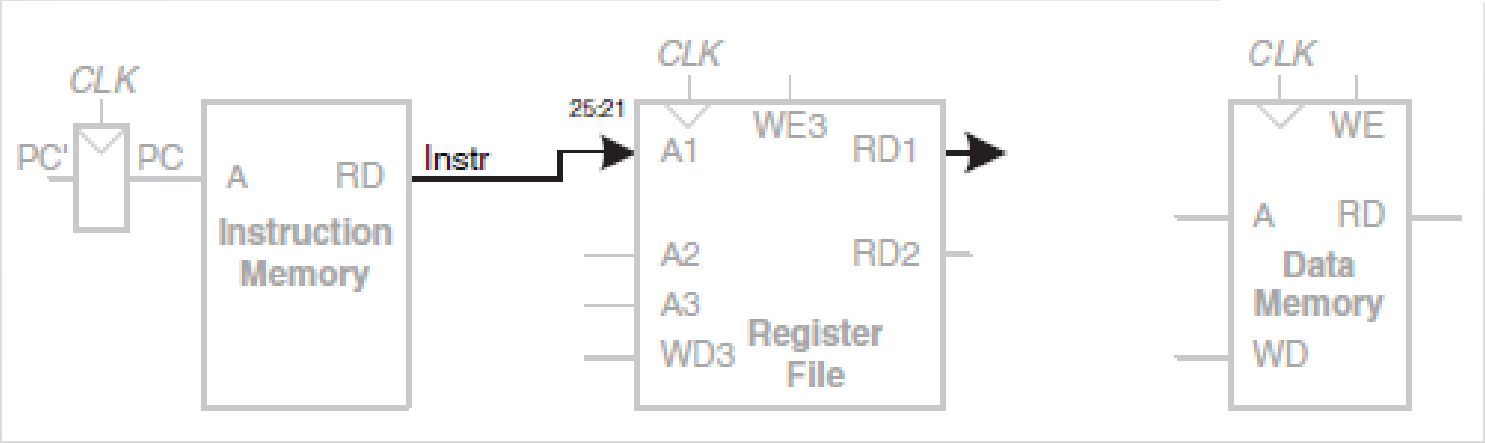
- The first step is to read this instruction from instruction memory.
- The PC is simply connected to the address input of the instruction memory.
- The instruction memory reads out, or fetches, the 32-bit instruction (Instr).
- The processor's actions depend on the specific instruction that was fetched.
- Construct the datapath connections for the **lw** instruction, then generalize it to handle the other instructions.



3. Single-Cycle Processor

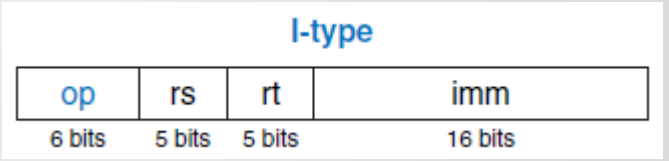
- For a **lw** instruction, the source register is read containing the base address.
- This register is specified in the rs field of the instruction, Instr[25:21].
- These bits of the instruction are connected to the address input of one of the register file read ports, A1.
- The register file reads the register value onto RD1.

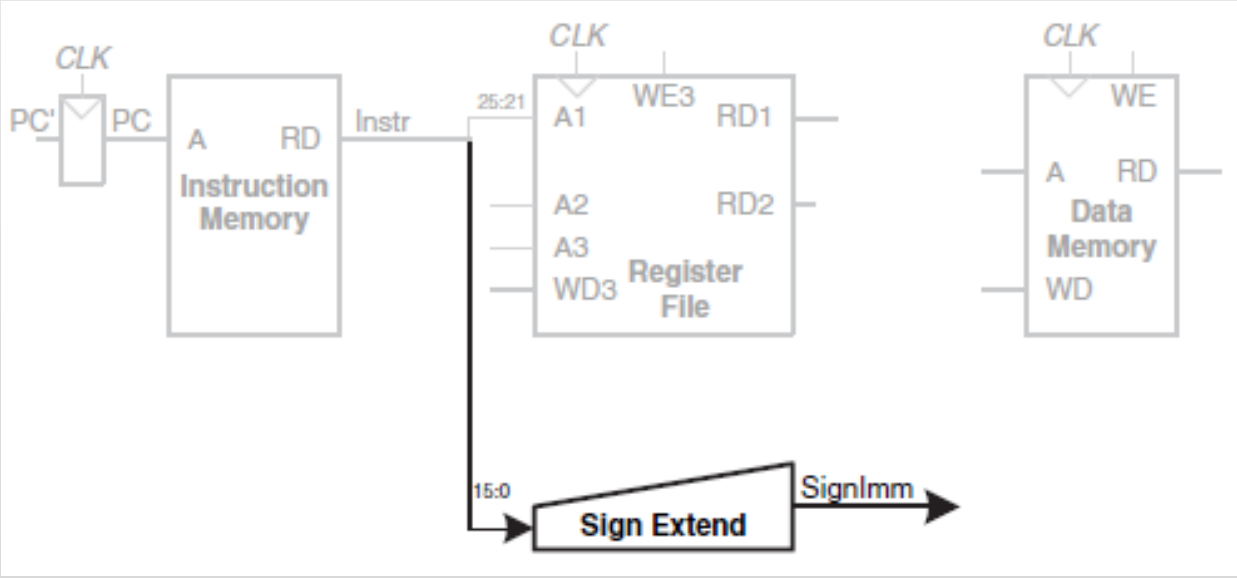




3. Single-Cycle Processor

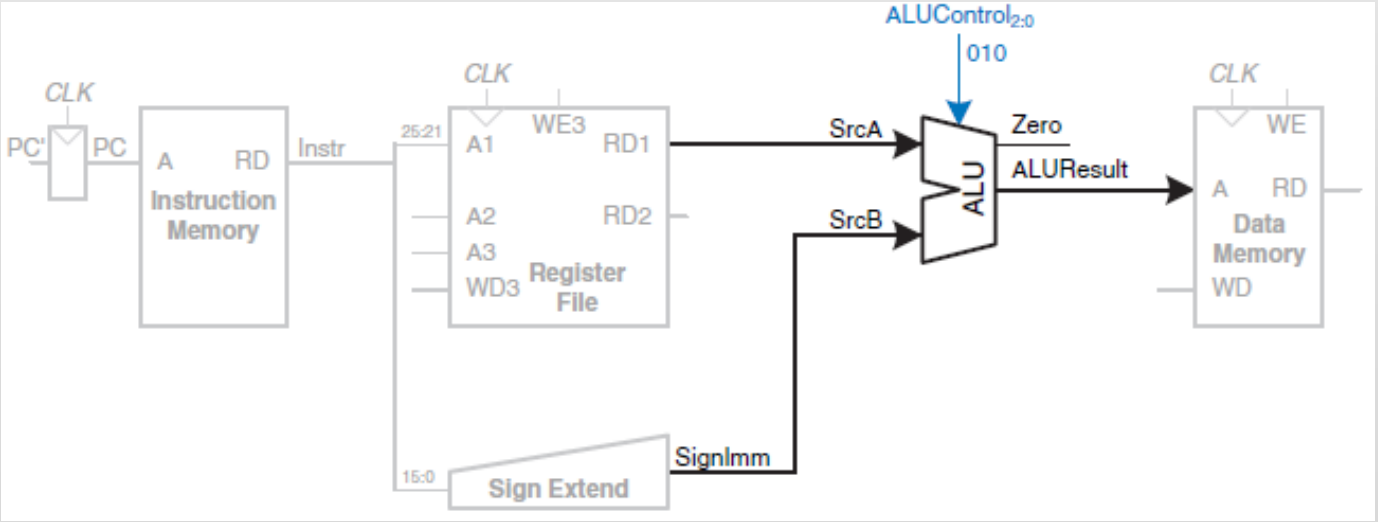
- The **lw** instruction also requires an offset which is stored in the immediate field of the instruction, Instr[15:0].
- Because the 16-bit immediate might be either positive or negative, it must be sign-extended to 32 bits **(SignImm)**.
  - SignImm [15:0] = Instr [15:0]
  - SignImm [31:16] = Instr [15].





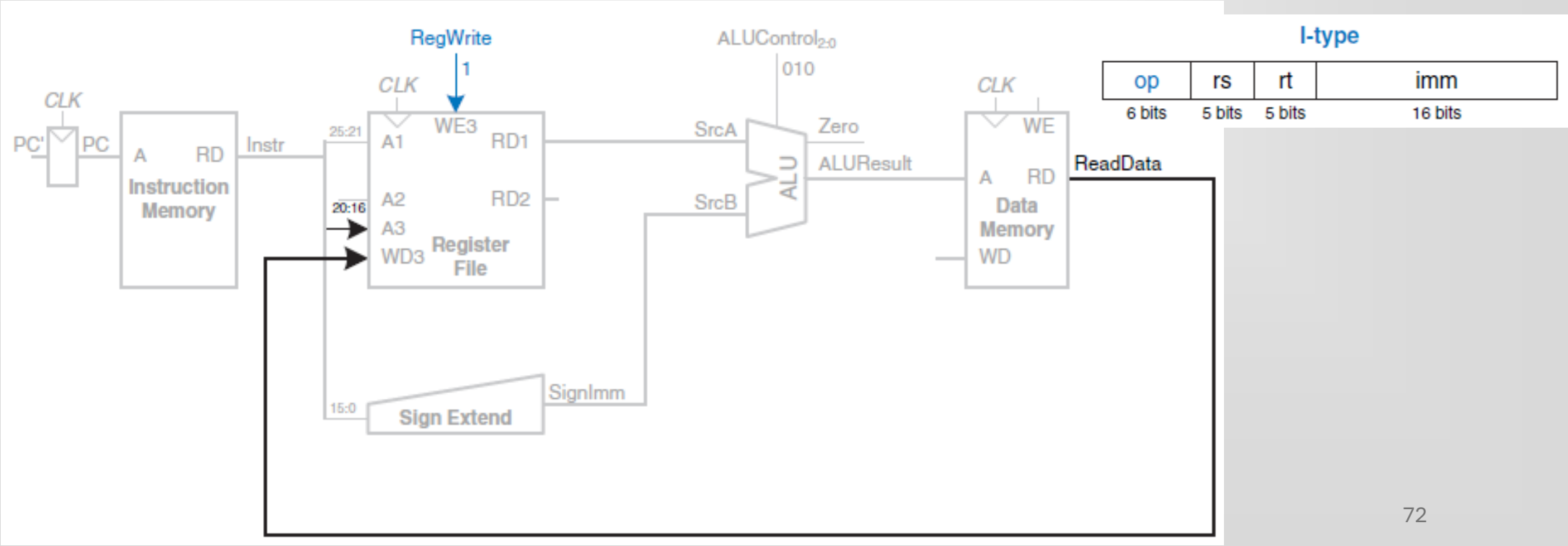
3. Single-Cycle Processor

- The processor adds the base address to the offset to find the read address from memory.
- The addition operation is performed using ALU which receives two operands, SrcA (from the register file) and SrcB (from the sign-extended immediate).
- The 3-bit ALUControl signal specifies the operation. The ALU generates a 32-bit ALUResult and a Zero flag, that indicates whether ALUResult == 0.
- For a **lw** instruction, the ALUControl is set to (010) to add the base address and offset.
- ALUResult is sent to the data memory as the address for the load instruction,



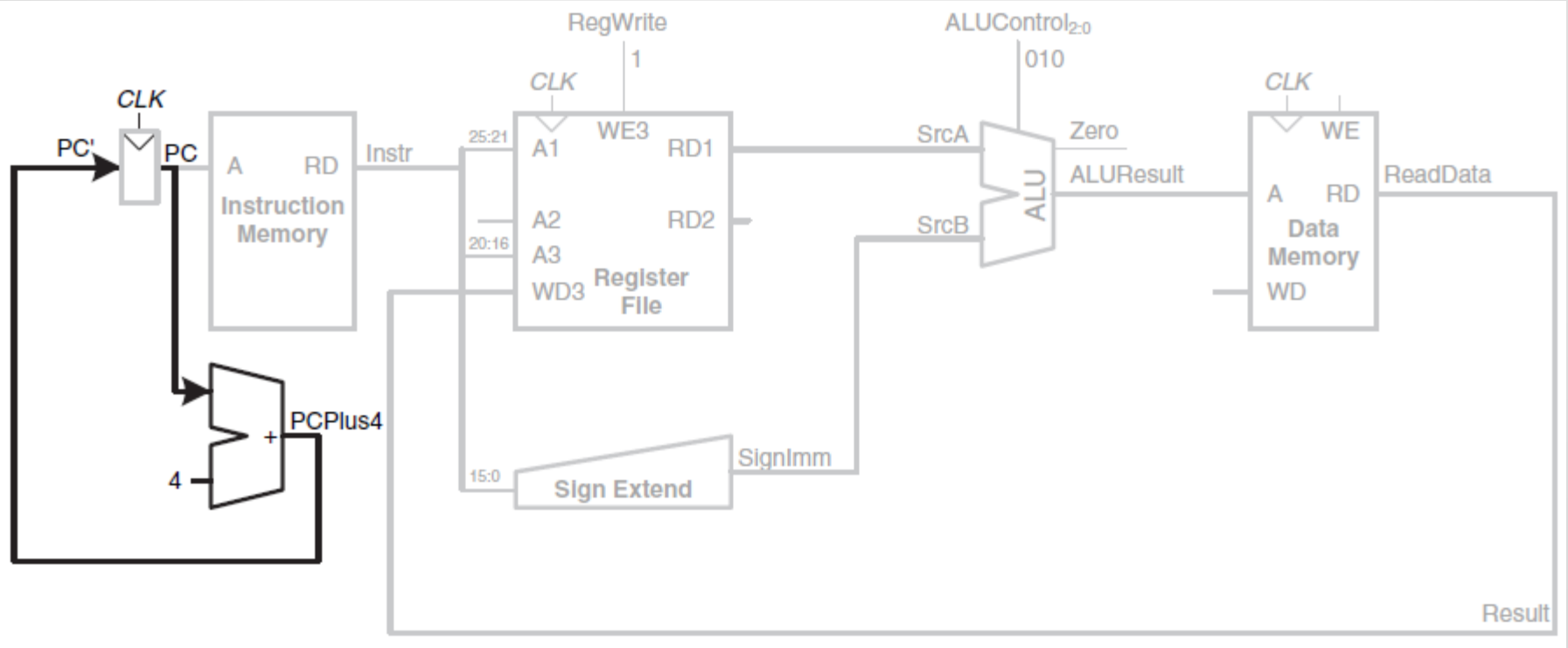
3. Single-Cycle Processor

- The destination register for the lw instruction is specified in the rt field, Instr [20:16], which is connected to the port 3 address input, A3, of the register file.
- The ReadData bus is connected to the port 3 write data input, WD3, of the register file.
- A control signal (RegWrite) is connected to the port 3 write enable input, WE3, and is asserted during a lw instruction so that the data value is written into the register file.



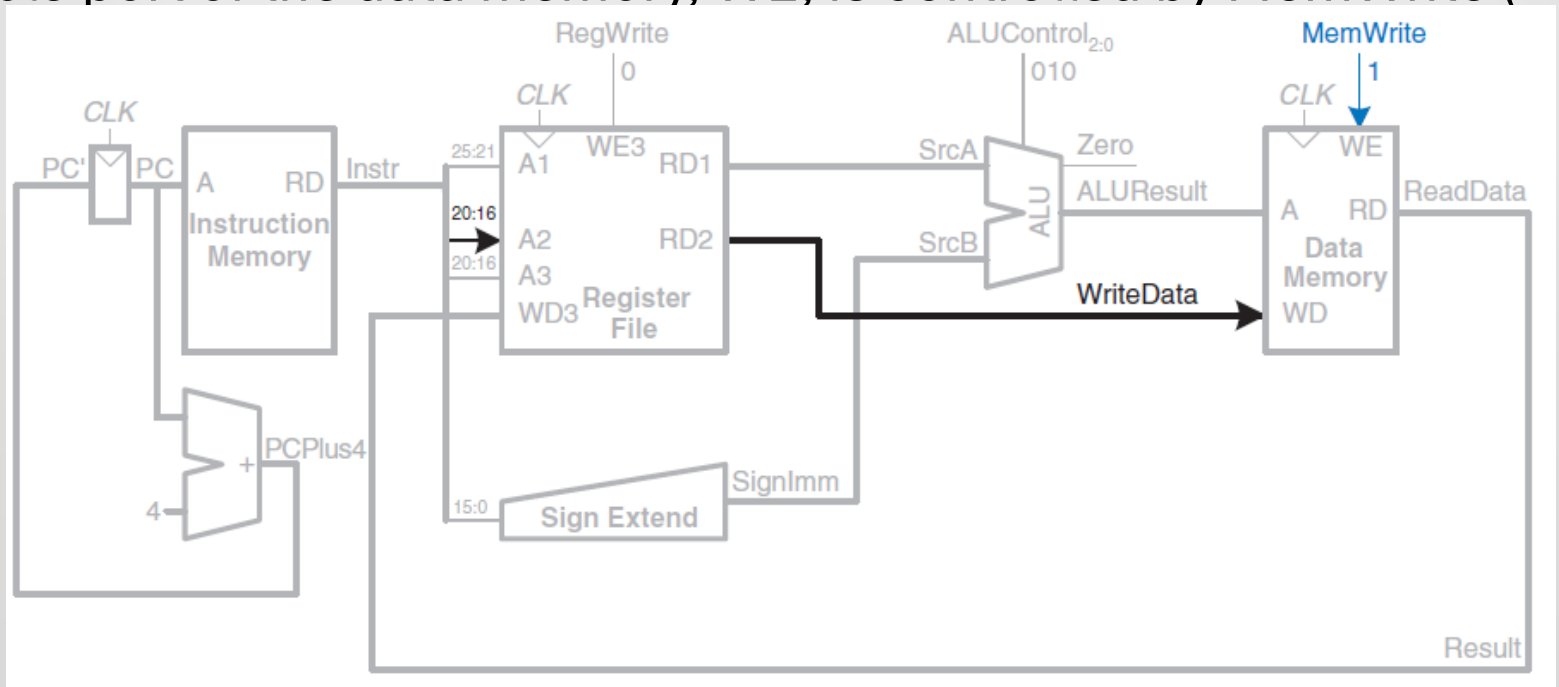
3. Single-Cycle Processor

- While the instruction is being executed, the processor must compute the address of the next instruction, PC'.
- Because instructions are 32 bits = 4 bytes, the next instruction is at PC + 4.
- Another adder is used to increment the PC by 4. The new address is written into the program counter on the next rising edge of the clock.



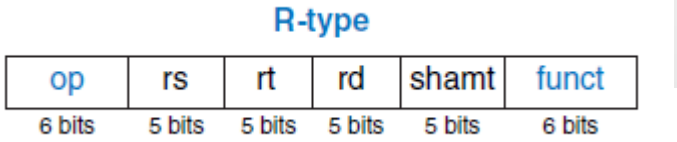
3. Single-Cycle Processor

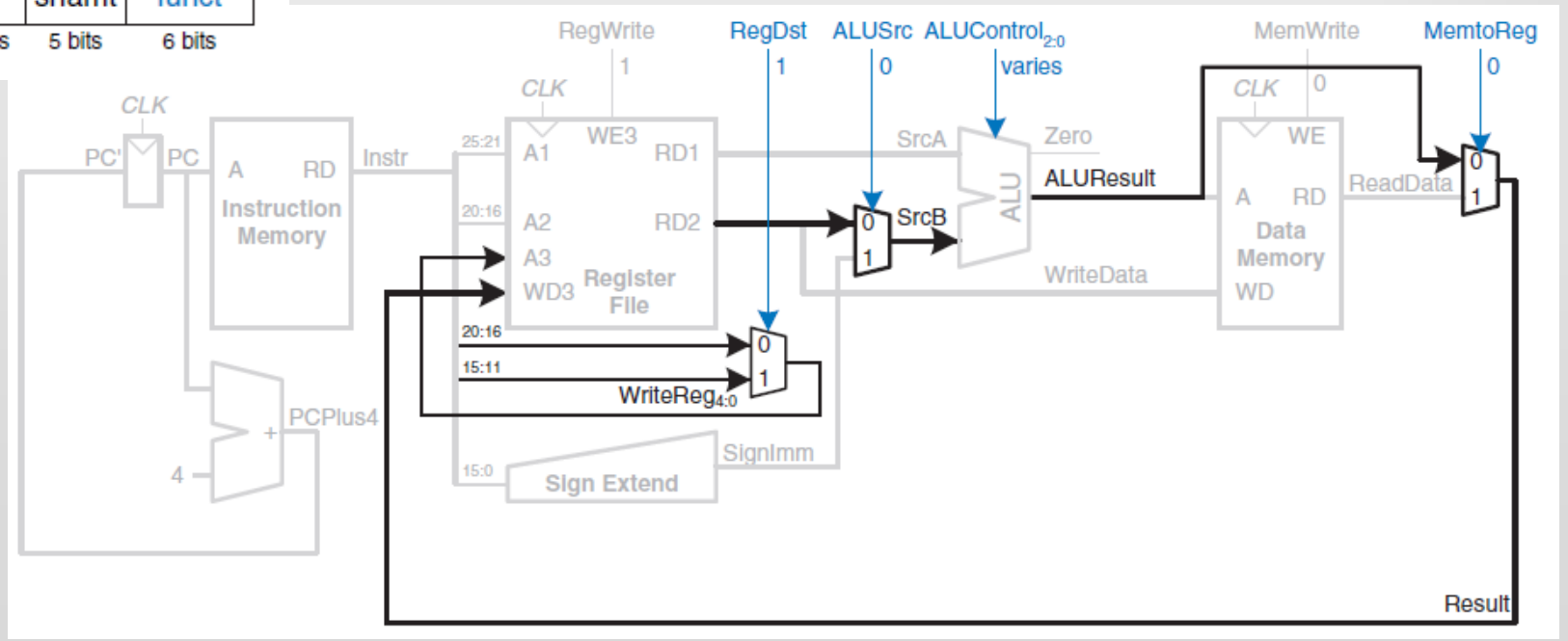
- To handle the sw instruction, we will reuse the already implemented HW for lw to calculate the write memory address.
- The sw instruction reads the write data from the register file which is specified in the rt field, Instr [20:16] to be connected to the second register file read port, A2.
- The output is read onto RD2 which is connected to the write data port of the data memory.
- The write enable port of the data memory, WE, is controlled by MemWrite (=1).



3. Single-Cycle Processor

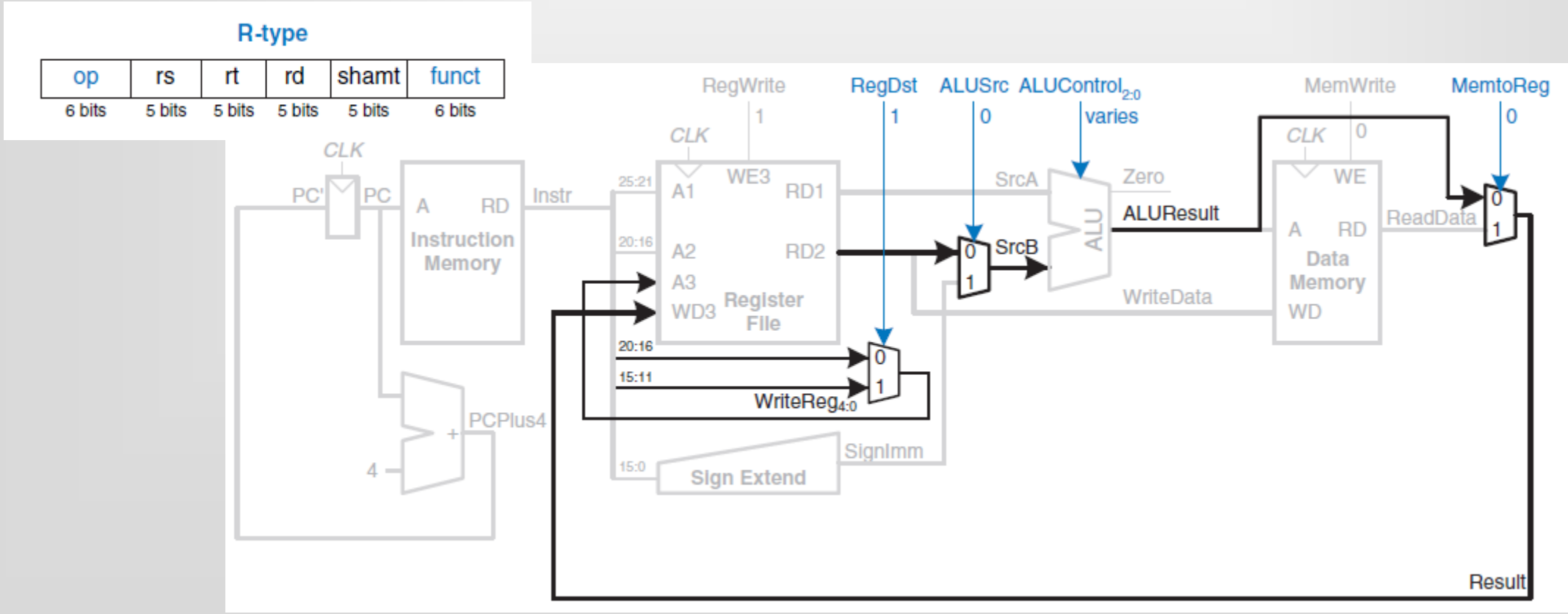
- To handle the R-type instructions add, sub, and, or, and slt:
- They differ only in the specific ALU operation. Hence, they can all be handled with the same hardware, using different ALUControl signals.
- A multiplexer is added to choose SrcB from either the register file RD2 port or SignImm, which is controlled by ALUSrc (0 for R-type and 1 for lw/sw).





3. Single-Cycle Processor

- Another multiplexer is added to choose between ReadData and ALUResult, which is controlled by, MemtoReg (0 for R-type and 1 for lw).
- The dest register is specified by the rd field, Instr [15:11]. Thus, we add a third multiplexer to choose WriteReg, which is controlled by RegDst (1 for R-type and 0 for lw)..

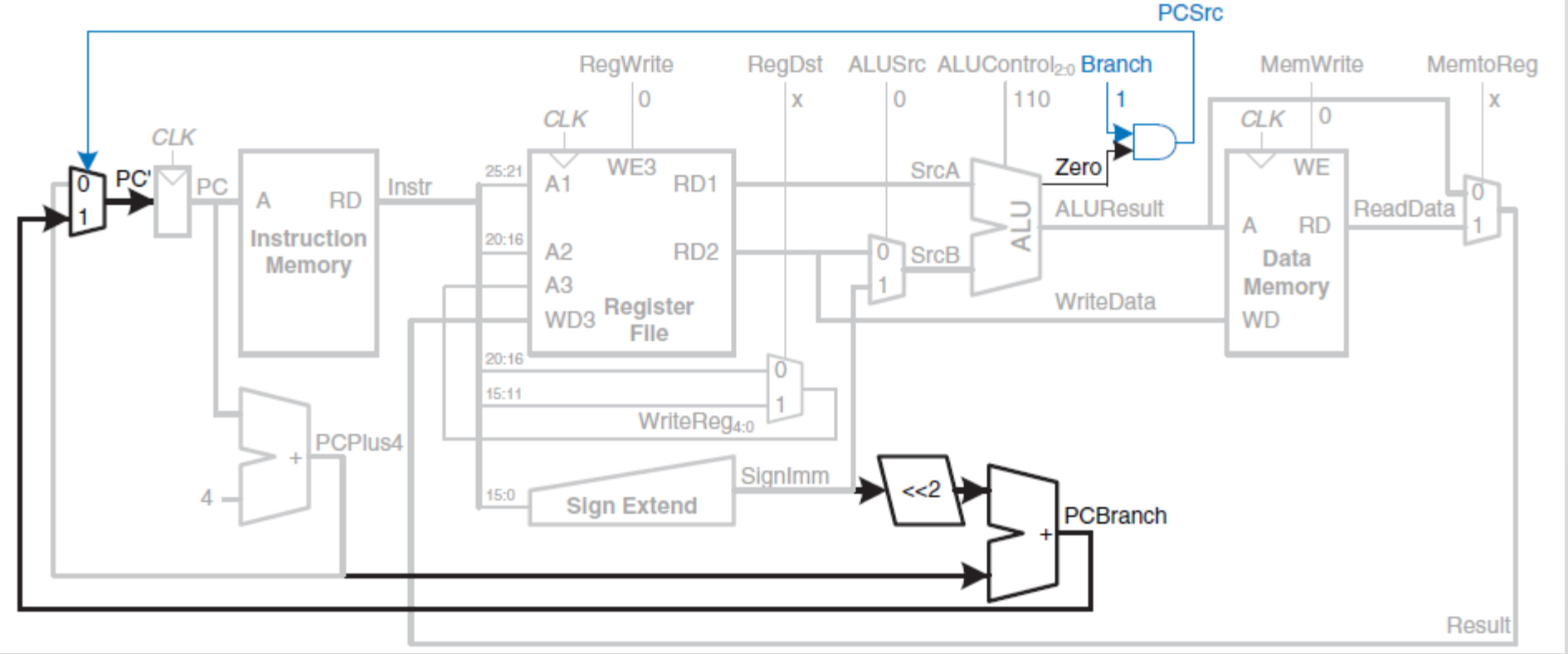


3. Single-Cycle Processor

I-type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

- To handle beq instruction:
- It compares (sub) two registers. If they are equal, it takes the branch by adding the branch offset to the program counter.
- The next PC value for a taken branch, PCBranch, is computed by sign-extending the immediate and multiplied by 4 (shifting SignImm left by 2 bits) then adding it to PCPlus4.



3. Single-Cycle Processor

- The two registers are compared by computing SrcA – SrcB using the ALU. If ALUResult is 0 (Zero flag=1), the registers are equal.
- We add a multiplexer to choose PC' from either PCPlus4 or PCBranch (a branch instruction, Zero flag = 1).
- For beq, the ALU performs a subtraction, so ALUControl = 110.
- ALUSrc = 0 to choose SrcB from the register file.
- RegWrite and MemWrite are 0, because it doesn't write to the register file or memory.
- We don't care about the RegDst and MemtoReg, because the register file isn't written.

3. Single-Cycle Processor

3.1 Single-Cycle Datapath

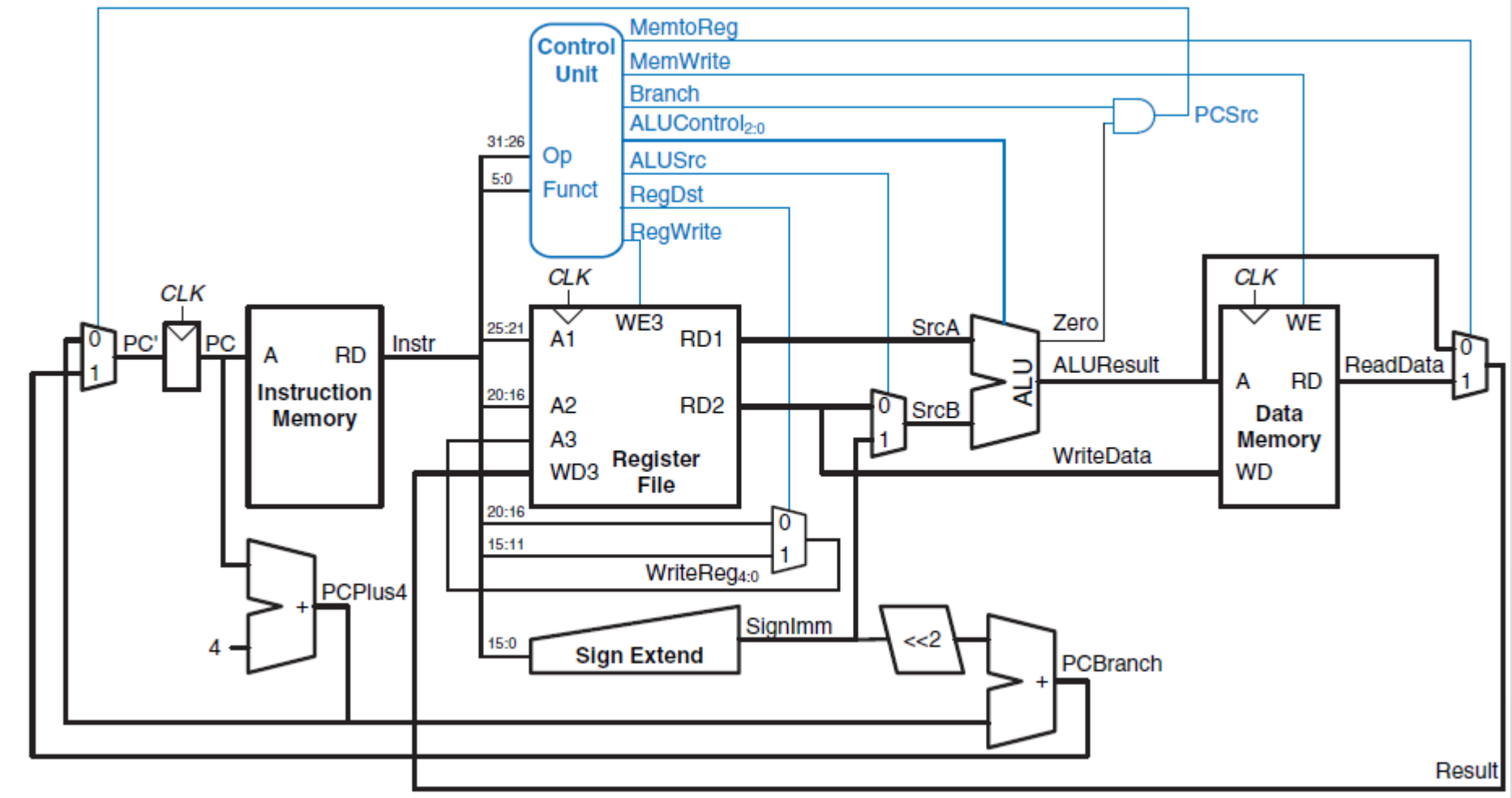
3.2 Single-Cycle Control

3.3 More Instructions

3.4 Performance Analysis

3. Single-Cycle Processor

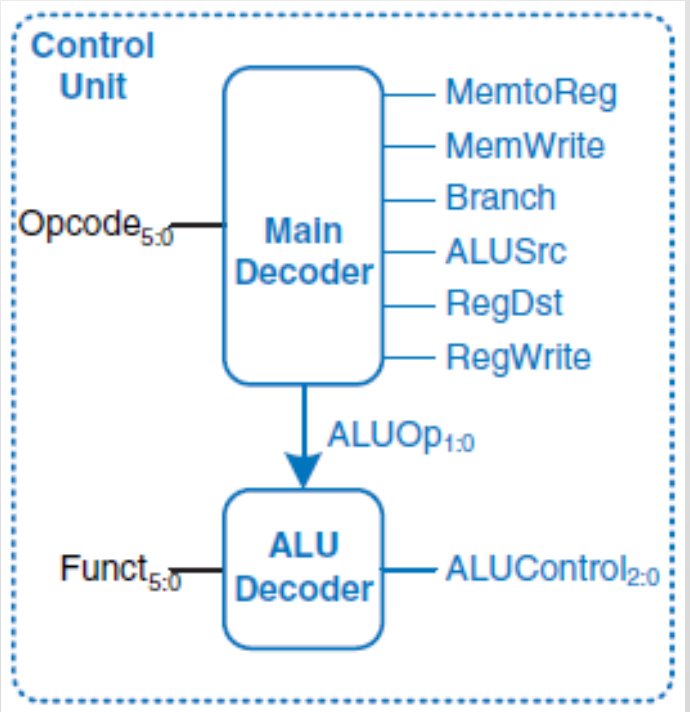
- The control unit computes the control signals based on the opcode (Instr [31:26]) and funct (Instr [5:0]) for R-type instructions, or the opcode only for the others.



3. Single-Cycle Processor

- Thus, we will simplify our design by dividing the control unit into two blocks of combinational logic.
- The main decoder computes most of the outputs from the opcode.
  It also determines a 2-bit ALUOp signal.
- The ALU decoder uses this ALUOp signal with the funct field to compute ALUControl.

ALUOp	Meaning
00	add
01	subtract
10	look at funct field
11	n/a



3. Single-Cycle Processor

- When ALUOp is 00 or 01, the ALU should add or subtract, respectively.
- When ALUOp is 10, the decoder examines the funct field to determine the ALUControl.
- The first two bits of the funct field for the R-type instructions are always 10, so we may ignore them to simplify the decoder.

F2:0	ALU Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND B
101	A OR B
110	A − B
111	SLT

ALUOp	Funct	ALUControl
00	X	010 (add)
X1	X	110 (subtract)
1X	100000 (add)	010 (add)
1X	100010 (sub)	110 (subtract)
1X	100100 (and)	000 (and)
1X	100101 (or)	001 (or)
1X	101010 (slt)	111 (set less than)

3. Single-Cycle Processor

- The main decoder controls the signals as a function of the opcode.
- All R-type instructions use the same main decoder values; they differ only in the ALU decoder output.
- The RegDst and MemtoReg are don't cares (X), and RegWrite is 0, for instructions that do not write to the register file (sw and beq).

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	x	1	0	1	x	00
beq	000100	0	x	0	1	0	x	01

3. Single-Cycle Processor

3.1 Single-Cycle Datapath

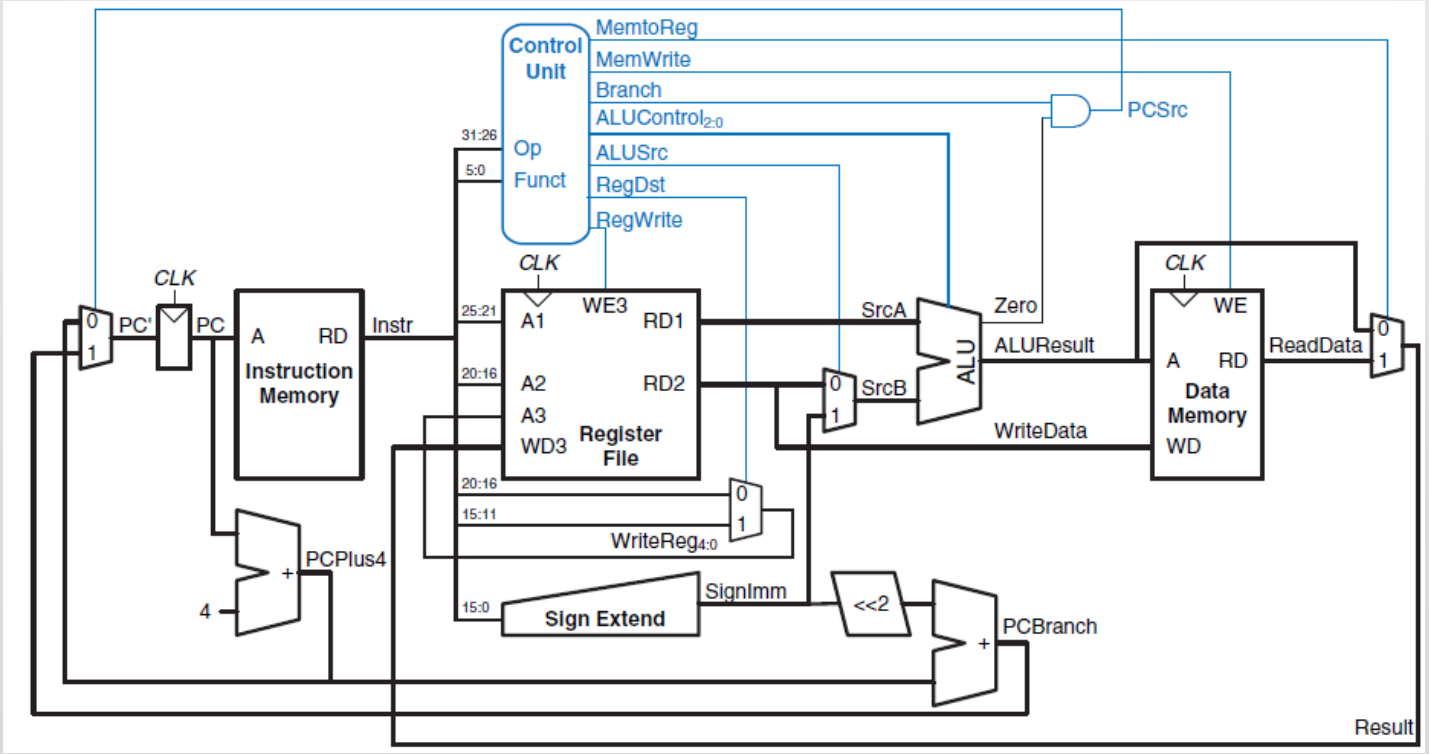
3.2 Single-Cycle Control

3.3 More Instructions

3.4 Performance Analysis

3. Single-Cycle Processor

- Supporting some instructions simply requires enhancing the main decoder, whereas supporting others also requires more hardware in the datapath.
- **Example:** The **addi** instruction adds the value in a register to the immediate and writes the result to another register.



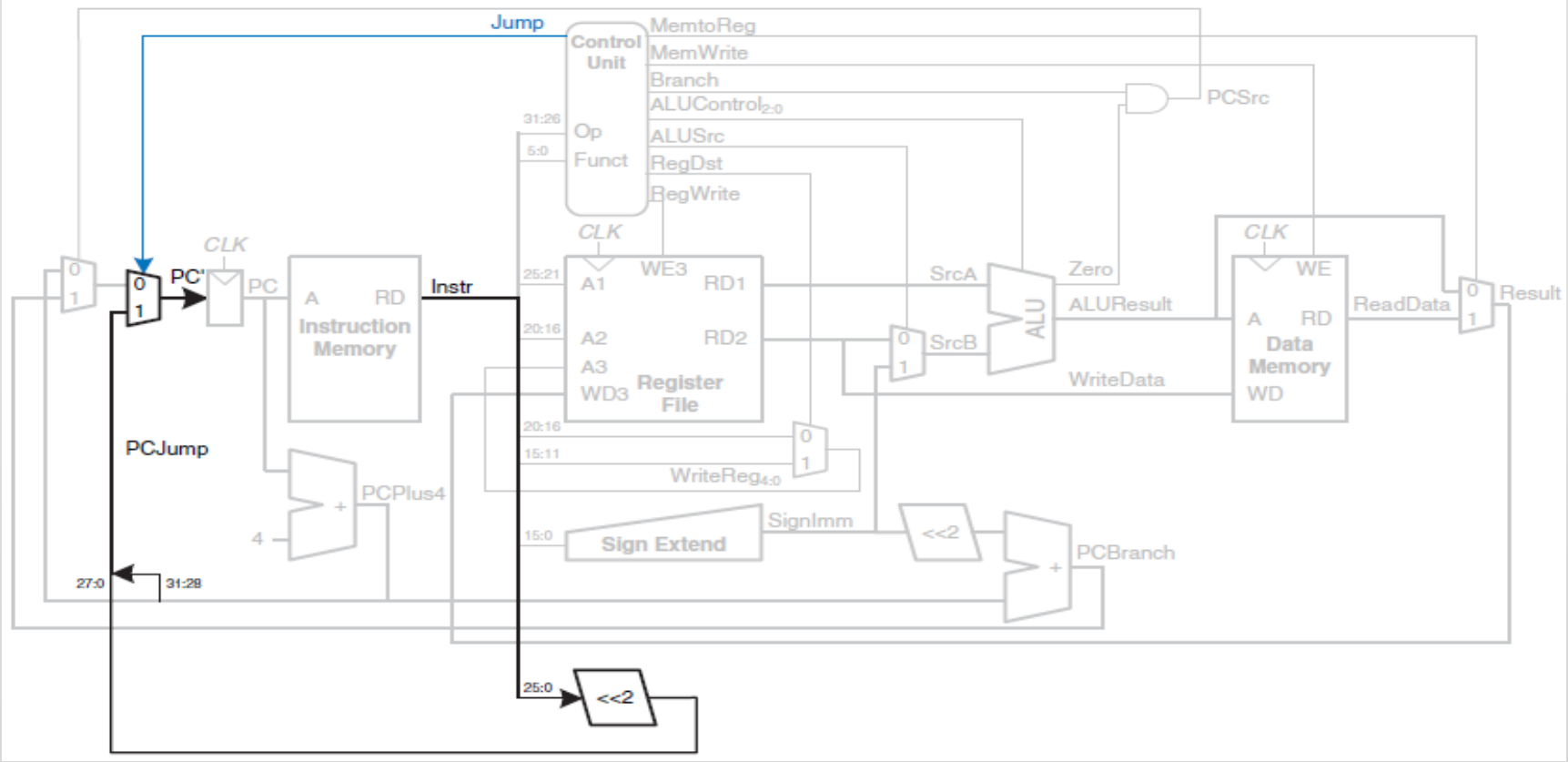
3. Single-Cycle Processor

- The result should be written to the register file, so RegWrite = 1.
- The destination register is specified in the rt field of the instruction, so RegDst = 0.
- SrcB comes from the immediate, so ALUSrc = 1.
- The instruction is not a branch, nor does it write memory, so Branch = MemWrite = 0.
- The result comes from the ALU, not memory, so MemtoReg = 0.
- Finally, the ALU should add, so ALUOp = 00.

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	x	1	0	1	x	00
beq	000100	0	x	0	1	0	x	01
addi	001000	1	0	1	0	0	0	00

3. Single-Cycle Processor

- **Example:** The jump instruction, **j**, writes a new value into the PC.
- We must add hardware to compute the next PC and a multiplexer to select this next PC:
  PC'[1:0] = 0, PC'[27:2] = Instr [25:0], PC'[31:28] = PC[31:28].
- The new multiplexer uses the new Jump control signal.



3. Single-Cycle Processor

- Main decoder TT must be enhanced to support the **j** instruction and the Jump signal.
- The Jump control signal is 1 for the j instruction and 0 for all others.
- j does not write the register file or memory, so RegWrite = MemWrite = 0.
- Hence, we don't care about the computation done in the datapath, and RegDst = ALUSrc = Branch = MemtoReg = ALUOp = x.

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	x	1	0	1	x	00	0
beq	000100	0	x	0	1	0	x	01	0
addi	001000	1	0	1	0	0	0	00	0
j	000010	0	x	x	x	0	x	xx	1

3. Single-Cycle Processor

3.1 Single-Cycle Datapath

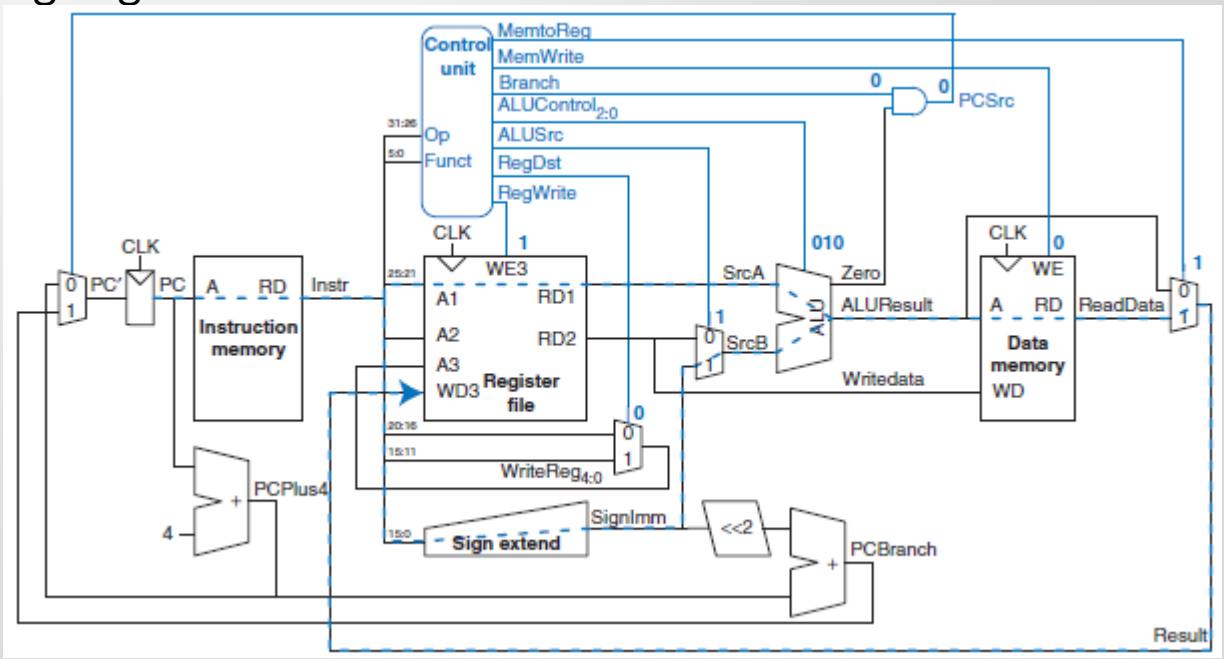
3.2 Single-Cycle Control

3.3 More Instructions

3.4 Performance Analysis

3. Single-Cycle Processor

- Each instruction in the single-cycle processor takes **one clock cycle**, so the **CPI** is 1.
- The critical path is the **lw** instruction.
  - It starts with the PC loading a new address on the rising edge of the clock.
  - The instruction memory reads the next instruction.
  - The register file reads SrcA. While the register file is reading, the immediate field is sign-extended and selected at the ALUSrc mux to determine SrcB.
  - The ALU adds SrcA and SrcB to find the address.
  - The data memory reads from this address.
  - The MemtoReg multiplexer selects ReadData.
  - Finally, Result must setup at the register file before the next rising clock edge, so that it can be properly written.



3. Single-Cycle Processor

- Hence, the cycle time is

$$T_c = t_{pcqPC} + t_{mem} + \max[t_{RFread}, t_{sext} + t_{mux}] + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

- In most implementation technologies, the ALU, memory, and register file accesses are substantially slower than other operations. Therefore, the cycle time simplifies to

$$T_c = t_{pcqPC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup}$$

- The numerical values of these times will depend on the specific implementation technology.
- Other instructions have shorter critical paths. For example, R-type instructions do not need to access data memory.
- The sequential design is synchronous, so the clock period is constant and must be long enough to accommodate the slowest instruction.

Assignment #1

Assignment #1

- Read section *7.3 SingleCycle Processor* from the textbook.
- Implement the **basic** SingleCycle MIPS Processor which supports only the mentioned 10 instruction set, documented in section *7.3*.
- Implement the rest MIPS blocks: ALU, register file, memory and multiplexers.
- Do a solid testbench using direct and randomized testcases.
- Deadline: Friday, 25th October 2024, at 12:00 PM.
- Deliveries: Design files, testbench, a document and a presentation for your work.

Thank You